Python Interview Questions & Answers

@python_world_in

1. Python Basics

1. What are python's key features?
Interpreted, high-level, dynamically typed, object-oriented, cross-platform, supports multiple paradigms, and has extensive libraries.

2. What are python's data types?
int, float, complex, str, list, tuple, set, dict, bool, None Type.

3. What is PEP8 and why it is important?
python's style guide; ensures consistent, clean, and readable code.

4. Explain mutable vs immutable objects.
Mutable (list, dict, set) can change.
Immutable (Tuple, str, int) cannot change.

5. What are python's built-in data structure?

List, tuple, set dictionary.

6. Difference between python2 and python 3

python 3 uses Unicode strategy by default print() function, better division handling, etc.

7. What are python's memory management features?

Private heap, reference counting, garbage collector.

8. Explain Indentation in python?

Indentation defines code blocks, mandatory unlike other languages.

9. How is python interpreted?

Compiled into bytecode, executed by Python virtual Machine (PVM).

10. Explain namespaces.

Mapping of variable names to objects (local, global, built-in).

2. Data structures

11. List vs Tuple differences.

List = mutable, slower, Tuple = immutable, faster, hashable.

12. How are sets used?

Store unique elements, fast membership tests.

13. What are dictionaries?

key-value pairs, unordered, fast lookups.

14. How to merge dictionaries?

{**dict1, **dict2} or dict1.update(dict2).

15. Remove duplicates from list?

list(set(lst)) or list(dict.fromkeys(lst))

16. What are list comprehensions,

compact syntax for lists: [x**2 for x in range(5)].

17. Flatten nested list?

sum(nested, []) or itertools.chain(*nested)

18. Shallow vs deep copy?

Shallow copies refs; deep copy clones objects recursively.

19. How slicing works?

list[start:end:step] creates sublist.

20. Reverse a list?

list[::-1] or reversed(lst).

21. frozenset in python?

Immutable set, hashable, can be dict keys.

22. Difference between is and ==?

is → identity, == → equality.

23. Sort dict by value?

sorted(d.items(), key = lambda x: x[1]).

24. Explain heapq.

Heap queue (priority queue) algorithms.

25 Implement stack and queue?

stack → list append/pop.

Queue → collections.deque

3. functions & OOP

26. function vs method?

function = independent

method = bound to object

27 * args and kwargs?

* args = variable positional args.

** kwargs = variable keyword args.

28 Recursion Example?

function calling itself (factorial, fibonacci)

29. What are decorators?

functions that wrap others to extend behavior.

30. What are generators?

functions with yield, return iterators, memory-efficient.

31. Iterator vs generator?

Iterator uses __iter__ / __next__;
generator auto creates them with yield.

32. With statement?

Ensures resource cleanup (e.g., file handling)

33. What are closures?

function with access to variables from enclosing scope.

34. Lambda functions?

Small anonymus functions: lambda x: x * 2.

35. @staticmethod vs @classmethod?

Staticmethod → no class / self ref.
classmethod → takes class as first arg.

36. Method resolution order (MRO)?

Defines method lookup order in inheritance hierarchy.

37. What is init?

Constructor method, initalizes object attributes.

38. Inheritance in python?

Reuse properties / methods from parent class.

39. Multiple -vs multi-level inheritance?

Multiple → multiple parents.

Multi-level → chain of inheritance

40. What is polymorphism?

Same method name with different implementations.

41. Abstraction?

Hiding implementation, exposing only essential details.

42. Encapsulation?

Building data & methods; restricts direct access.

43. Magic methods?

Special methods like __add__, __len__, __str__.

44. str vs repr?

str → readable. repr → unambiguous (debugging).

45. Monkey patching?

Dynamically changing class / module behavior at runtime.

4. Exception Handling

46. Errors vs Exceptions?

Errors = compile / runtime issues.

Exceptions = events handled via try-except

47. Try-except-finally?

Try → code, Except → handle,

finally → always executes.

48. Raise vs assert?

Raise → throw exception. Assert - test condition.

49. Custom exceptions?

Subclass Exception to define custom error.

50. Exception chaining?

One exception caused by another (raise... from....).

51. Syntax vs runtime error?

Syntax → invalid code.

Runtime → occurs during execution.

52. Catch multiple exceptions?

except (TypeError, ValueError):.

53. Use of else in exception Handling?

Executes only if no exception in try block.

54. If exception not handled?

program terminates with traceback.

55. Pass vs continue in exceptions?

Pass = do nothing. Continue = move to next iteration.

follow for More...!!

@python_world_in